SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	KS THARUNESHWAR	**Reg. No.:**	192572148
Section / Batch:	2025	**Date:**	22-08-2026

Instructions to Students

- Answer both questions carefully. Each question carries 50 marks. Total: 100 marks.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Description	Questions	Total Marks
Comparative Analysis	Focus on Comparison of different Algorithms using dynamic programming and greedy strategies.	Q1 – Q2	100

SECTION A – Comparative Analysis

A)Knapsack Problem vs Container Loading (DP vs Greedy)

Given weights and capacity, solve using 0/1 Knapsack (DP) and Container Loading (Greedy). Compare selected items and utilization efficiency. Analyze why greedy fails for discrete optimization. Evaluate computational complexity. Discuss real-world logistics applications. Generate insights on solution strategies.

1. Define the Word Break and Longest Palindromic Subsequence problems with suitable examples.

The **Word Break Problem** determines whether a given string can be divided into valid words present in a

dictionary.

Example: String: applepie, Dictionary: {apple, pie} applepie

= apple + pie

Longest Palindromic Subsequence (LPS): LPS finds the longest subsequence of a string that is a palindrome.

Example:
String: BBABCBCAB
Longest Palindromic Subsequence: BABCBAB

2. Write the DP algorithm or pseudocode.

Word Break Algorithm :

```
dp[0] = true

for i = 1 to n
   for j = 0 to i-1
      if dp[j] is true and s[j...i-1] is in dictionary
dp[i] = true
         break

return dp[n]
```

Longest Palindromic Subsequence :

```
for i = 0 to n-1
      dp[i][i] = 1

for length = 2 to n
for i = 0 to n-length
      j = i + length - 1

      if s[i] == s[j]
         dp[i][j] = dp[i+1][j-1] + 2
      else
         dp[i][j] = max(dp[i+1][j], dp[i][j-1])

return dp[0][n-1]
```

3. Prepare the DP table/state representation.

Word Break:

String: "applepie"

i	Prefix	dp[i]
0	""	True

1	a	False
2	ap	False
3	app	True
4	appl	False
5	apple	True
8	applepie	True

Longest Palindromic Subsequence:

	B	**B**	**A**	**B**	**C**
B	1	2	2	3	3
B		1	1	2	2
A			1	1	1
B				1	1
C					1

4. Develop the recurrence relation

Word Break

dp[i]=true if there exists a j < i such that:
dp[j]=true
and
s[j...i−1] is present in the dictionary.

LPS

If:
s[i]=s[j] then:
dp[i][j]=dp[i+1][j−1]+2
Otherwise:
dp[i][j]=max(dp[i+1][j],dp[i][j−1]) Base case:
dp[i][i]=1

5. Implement the Longest Palindromic Subsequence using Dynamic Programming (Saveetha Compiler

6. **Analyze time complexity**

Problem	**Time Complexity**
Word Break	**O(n²)**
LPS	**O(n²)**

7. **Analyze space complexity.**

Problem	**Space Complexity**
Word Break	**O(n)**
LPS	**O(n²)**

8. **Compare the two DP formulations using a table.**

Feature	**Word Break**	**LPS**
DP State	dp[i]	dp[i][j]
State Type	1D	2D
Purpose	Check valid segmentation	Find longest palindrome
Base Case	dp[0] = true	dp[i][i] = 1
Transition	Check possible word splits	Compare s[i] and s[j]
Output	Boolean	Integer
Time	O(n²)	O(n²)
Space	O(n)	O(n²)

9. **Identify similarities between the two problems.**

- Both are **Dynamic Programming problems**.

• Both deal with **strings**.
• Both divide the problem into smaller subproblems.
• Both store previously calculated results.
• Both avoid repeated computation.
• Both have **O(n²) time complexity** in their standard DP solutions.

10. Identify differences in state definition and transitions.

Word Break

- Uses a 1D state dp[i].
- Focuses on prefixes and partition points.
- Checks whether a substring is a valid dictionary word.

LPS

- Uses a 2D state dp[i][j].
- Focuses on ranges/intervals of the string.
- Compares characters at both ends and chooses the maximum.

11. Compare the DP state, transition, implementation, and complexity of both problems and provide a conclusion.

Item	Weight	Value
A	2	20
B	5	30
C	7	40
D	3	10

Comparision:

Feature	0/1 Knapsack – DP	Container Loading – Greedy
Approach	Dynamic Programming	Greedy
State	dp[i][w]	Current remaining capacity
State Meaning	Maximum value using first i items with capacity w	Capacity available after selections
Transition	max(take, not take)	Select next best feasible item
Implementation	2D DP table	Sorting + selection
Time	O(nW)	O(n log n)
Space	O(nW)	O(n)

Optimality	Always optimal	Not always optimal
Advantage	Gives best combination	Faster and simpler
Disadvantage	More time and memory	Can miss optimal combination

Conclusion:

0/1 Knapsack using Dynamic Programming **is better when we need the** optimal value and combination of items**.** Container Loading using Greedy **is faster and simpler, but it may not always give the optimal solution because it makes local decisions. Therefore,** DP is preferred for discrete optimization problems where accuracy and optimality are more important, while Greedy is preferred when fast approximate solutions are acceptable.

B) Travelling Salesperson Problem Comparison (DP vs Greedy Heuristic)

Given 4 cities and distance matrix, solve TSP using DP and nearest neighbor greedy approach. Compare total tour cost and computational effort. Analyze why greedy does not guarantee optimality. Evaluate exponential complexity of DP. Discuss scalability challenges. Generate insights on exact vs heuristic methods

1. Define the Word Break and Longest Palindromic Subsequence problems with suitable examples.

The **Word Break Problem** determines whether a given string can be divided into valid words present in a dictionary.

Example: String: applepie, Dictionary: {apple, pie} applepie

= apple + pie

Longest Palindromic Subsequence (LPS): LPS finds the longest subsequence of a string that is a palindrome.

Example:
String: BBABCBCAB
Longest Palindromic Subsequence: BABCBAB

2. Write the DP algorithm or pseudocode.

Word Break Algorithm : dp[0]
= true

```
for i = 1 to n
   for j = 0 to i-1
      if dp[j] is true and s[j...i-1] is in dictionary
dp[i] = true            break

return dp[n]
```

Longest Palindromic Subsequence :

```
for i = 0 to n-1
      dp[i][i] = 1

for length = 2 to n
for i = 0 to n-length
      j = i + length - 1

      if s[i] == s[j]
         dp[i][j] = dp[i+1][j-1] + 2
      else
         dp[i][j] = max(dp[i+1][j], dp[i][j-1])

return dp[0][n-1]
```

3. Prepare the DP table/state representation.

Word Break:

String: "applepie"

i	Prefix	dp[i]
0	""	True
1	a	False
2	ap	False
3	app	True
4	appl	False
5	apple	True
8	applepie	True

Longest Palindromic Subsequence:

	B	B	A	B	C
B	1	2	2	3	3

B		1	1	2	2
A			1	1	1
B				1	1
C					1

4. Develop the recurrence relation

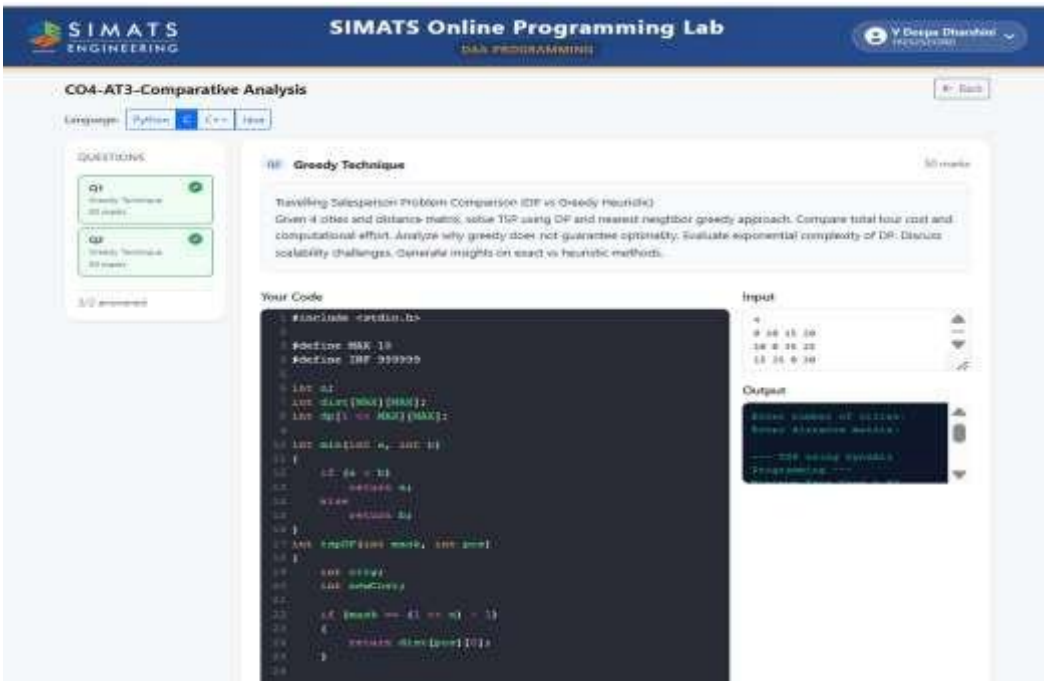
Word Break

dp[i]=true if there exists a j < i such that: dp[j]=true and s[j...i−1] is present in the dictionary.

LPS

If:
s[i]=s[j] then:
dp[i][j]=dp[i+1][j−1]+2
Otherwise:
dp[i][j]=max(dp[i+1][j],dp[i][j−1]) Base case:
dp[i][i]=1

5. Implement the Longest Palindromic Subsequence using Dynamic Programming (Saveetha Compiler)



6.Analyze time complexity

Problem	Time Complexity
DP	**O(nW)**
Greedy	**O(n log n)**

7.Analyze space complexity.

Problem	Space Complexity
DP	**O(nW)**
Greedy	**O(n)**

8.Compare the two DP formulations using a table.

Feature	0/1 Knapsack	Container Loading
Approach	DP	Greedy
State	dp[i][w]	Remaining capacity
State Type	2D	No DP table
Purpose	Maximize total value	Maximize container utilization
Decision	Take or don't take	Select next suitable item
Transition	Maximum of take/not take	Choose locally best item
Output	Maximum value	Selected items/utilization
Time	O(nW)	O(n log n)
Space	O(nW)	O(n)

9.Identify similarities between the two problems.

- Both deal with **limited capacity**.
- Both involve **selecting items**.
- Both consider **item weights**.
- Both aim to use the available capacity efficiently.
- Both can be applied to **logistics and resource allocation**.
- Both try to obtain a good solution under capacity constraints.

10.Identify differences in state definition and transitions.

0/1 Knapsack

- Uses a **2D DP state** dp[i][w].
- Focuses on the first i items and available capacity w.
- Checks whether to **include or exclude** an item.
- Stores previous results and chooses the maximum value.

Container Loading

- Uses **remaining capacity** instead of a DP state.
- Focuses on selecting the next suitable item.
- Makes a **local greedy choice**.
- Does not consider all possible combinations.

11.Compare the DP state, transition, implementation, and complexity of both problems and provide a conclusion.

Feature	**0/1 Knapsack – DP**	**Container Loading – Greedy**
DP State	dp[i][w]	No DP state
Transition	Take or don't take	Select next best item
Implementation	DP table	Sorting + greedy selection
Time	O(nW)	O(n log n)
Space	O(nW)	O(n)
Optimality	Guaranteed optimal	Not guaranteed
Advantage	Finds best combination	Faster and simpler
Disadvantage	More time and memory	May miss optimal solution

Conclusion:

- **Knapsack DP** considers different combinations and gives an **optimal solution**.
- **Container Loading Greedy** makes quick local decisions and is **faster**.
- Greedy may fail to find the best combination.
- Therefore, **DP is preferred when optimality is important, while Greedy is preferred when speed and simplicity are important.**

Code of Conduct

I V Deepa Dharshini (192525213)) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: KS THARUNESHWAR

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Mark Evaluation:

Questions	Selection of Studies/Parameters (20)	Comparison & Differentiation (25)	Critical Evaluation (25)	Synthesis & Insight Generation (20)	Clarity (10)	Total Marks (Each - 50)
Q1						
Q2						

Overall Total	

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____